

MIP_PHASE_0_TO_4_EXECUTION_GUIDE.md

Mainframe Intelligence Platform (MIP)

Beginner Friendly Execution Guide

Version 1.0

Read This First

Do NOT try to build:

- AI Agents
- Chatbots
- RAG
- LLM Integrations
- Business Rule Engines

Your objective is simple:

Convert mainframe source code into structured metadata and relationships.

Think:

Source Code ↓ Metadata ↓ Relationships ↓ Knowledge Graph

Everything else comes later.

Technology Stack

Python 3.13

VS Code

GitHub Copilot

FastAPI

SQLite

NetworkX

Pydantic v2

Pytest

Google ADK (later)

Phase 0

Foundation

Goal

Create a local development environment.

Step 1

Create Workspace

```
mip-platform/  
├─ mfcodes/  
├─ src/  
├─ tests/  
├─ docs/  
├─ output/  
├─ skills/  
└─ prompts/
```

Step 2

Create Python Environment

```
python -m venv .venv
```

Activate environment.

Step 3

Install Packages

```
pip install fastapi
pip install pydantic
pip install networkx
pip install pytest
pip install uvicorn
```

Step 4

Create Git Repository

```
git init
```

Commit initial structure.

Step 5

Load Mainframe Source

Copy source code into:

```
mfcode/
```

Example:

```
mfcode/
├── cobol/
├── jcl/
├── copybooks/
├── db2/
└── docs/
```

Success Criteria

VS Code opens project successfully.

Python environment works.

Source repository available locally.

Phase 1

Discovery

Goal

Answer:

What exists?

What You Build

Repository Scanner

Artifact Classifier

Inventory Generator

Step 1

Scan Repository

Read all files recursively.

Store:

```
{  
  "path": "...",  
  "name": "...",  
  "extension": "..."  
}
```

Step 2

Classify Files

Example:

```
.cbl  
.cob  
.cpy
```

```
.jcl  
.sql
```

Map to:

```
COBOL  
COPYBOOK  
JCL  
DB2  
UNKNOWN
```

Step 3

Generate Inventory

Create:

artifact_inventory.json

Example:

```
{  
  "type": "COBOL",  
  "count": 1450  
}
```

Step 4

Persist To SQLite

Create table:

artifacts

Columns:

```
id  
path  
name  
type
```

Copilot Prompt

Act as a Principal Python Architect.

Design a scalable repository discovery service.

Requirements:

- Recursive scanning
- SQLite persistence
- Extensible file classification
- Unit testing

Provide architecture only.

Success Criteria

Answer:

How many COBOL programs exist?

How many JCL files exist?

How many copybooks exist?

Phase 2

Program Relationship Discovery

Goal

Answer:

Who calls whom?

What You Build

CALL Parser

CALL Relationship Store

Call Graph

Mainframe Concept

Example:

```
CALL 'PAYUPD'
```

Meaning:

Current Program

calls

PAYUPD

Step 1

Extract PROGRAM-ID

Example:

```
PROGRAM-ID. PAYPOST.
```

Store:

```
PAYPOST
```

Step 2

Extract CALL Statements

Example:

```
CALL 'PAYUPD'
```

Store:

```
PAYPOST -> PAYUPD
```

Step 3

Create SQLite Tables

programs

call_relationships

Step 4

Build NetworkX Graph

Node:

Program

Edge:

CALLS

Example:

```
PAYPOST
  CALLS
PAYUPD
```

Step 5

Create Query

Questions:

```
Who calls PAYUPD?

What does PAYPOST call?
```

Copilot Prompt

```
Act as a Mainframe Architect.

Design a COBOL call relationship discovery framework.
```


Requirements:

- PROGRAM-ID extraction
- CALL extraction
- SQLite persistence
- NetworkX integration

Architecture only.

Success Criteria

Generate complete CALL graph.

Phase 3

Copybook Intelligence

Goal

Answer:

What shared structures exist?

Mainframe Concept

Example:

```
COPY CARDREC.
```

Meaning:

Program uses shared structure CARDREC.

What You Build

Copybook Parser

Copybook Catalog

Usage Relationships

Step 1

Extract COPY Statements

Example:

```
COPY CARDREC .
```

Store:

```
PAYPOST  
USES  
CARDREC
```

Step 2

Catalog Copybooks

Store:

```
CARDREC  
CUSTREC  
ACCTREC
```

Step 3

Create Relationship Table

program_copybook_relationships

Step 4

Add To NetworkX

Example:

PAYPOST
USES
CARDREC

Questions Answered

Which programs use CARDREC?

Which copybooks does PAYPOST use?

Copilot Prompt

Design a copybook discovery framework.

Requirements:

- COPY statement extraction
- Relationship persistence
- Graph integration

Architecture only.

Success Criteria

Generate Program → Copybook relationships.

Phase 4

DB2 Data Lineage

Goal

Answer:

What data is being used?

Mainframe Concept

Example:

```
EXEC SQL  
  
SELECT *  
FROM CUSTOMER  
  
END-EXEC
```

Meaning:

Program reads CUSTOMER table.

What You Build

DB2 Parser

Table Catalog

Lineage Relationships

Step 1

Extract Tables

Detect:

```
SELECT  
INSERT  
UPDATE  
DELETE
```

Step 2

Store Relationships

Examples:

PAYPOST
READS
CUSTOMER

PAYPOST
UPDATES
ACCOUNT

Step 3

Create SQLite Tables

tables

program_table_relationships

Step 4

Add To NetworkX

Example:

PAYPOST
UPDATES
ACCOUNT

Questions Answered

Which programs update ACCOUNT?

Which programs read CUSTOMER?

Copilot Prompt

Act as a DB2 and COBOL expert.

Design a SQL metadata extraction framework.

Requirements:

- EXEC SQL parsing
- Table lineage extraction
- SQLite persistence
- NetworkX integration

Architecture only.

Success Criteria

Generate Program → Table lineage.

Expected Output After Phase 4

Inventory

Programs

Jobs

Copybooks

Tables

CALL Relationships

COPY Relationships

READ Relationships

UPDATE Relationships

Example Questions You Can Answer

Which jobs execute PAYPOST?

Who calls PAYUPD?

What programs use CARDREC?

Which programs update ACCOUNT?

Which programs read CUSTOMER?

Show dependency chain for PAYPOST.

Stop Here

Do NOT start:

- LLMs
- RAG
- Agents
- Google ADK
- Business Rules

Until Phase 0–4 are fully working on a real repository.

Why This Matters

After Phase 4 you will have:

Source Code ↓ Metadata ↓ Relationships ↓ Knowledge Graph

This becomes the foundation for every future capability:

- Impact Analysis
- Reasoning
- AI Copilot
- Modernization Intelligence
- Service Discovery
- API Discovery

Without this foundation, every future feature becomes unreliable.

With this foundation, MIP becomes a true Enterprise Legacy Intelligence Platform.